

Yenepoya (Deemed to be University)

Project – Low Level Design

on

AI Self-Healing Cloud for Retail

Student Name:

Mohammed Raed (23BCAICD064)

Industry Mentor: _____, **Guided by:** [Guide
Name]

TABLE OF CONTENTS

1. INTRODUCTION

1.1 Scope Of The Document

1.2 Intended Audience

2. MODULE OVERVIEW

3. ALGORITHMIC DESIGN

3.1 Thread Configuration

3.2 Daemon Process Flow

3.3 Error Handling Failsafes

4. FUNCTION & CLASS CATALOGUE

5. DATABASE SCHEMA & EXECUTION

5.1 Table Definitions

5.2 SQL Execution Patterns

6. NETWORK & HARDWARE INTERFACES

7. VARIABLE & STATE MUTABILITY

8. MEMORY OPTIMIZATION

9. RUNTIME SECURITY CONSTRAINTS

10. REFERENCES

1. INTRODUCTION

1.1 Scope Of The Document

This document specifies the rigorous, low-level programmatic structures, algorithmic logic gates, and explicit code-level executions of the AI Self-Healing Cloud application script (`monitor.py`). Unlike the High-Level Design which focuses on macro-architecture and conceptual data flow, this Low-Level Design (LLD) is explicitly dedicated to translating those concepts into functional software methodologies. The scope covers the precise utilization of Python standard libraries, thread memory management, specific parameterization of SQL cursors, and the exact exception-handling blocks mandated to prevent runtime degradation.

1.2 Intended Audience

This document is authored exclusively for backend Python developers, database administrators, system reliability engineers, and faculty technical evaluators. It requires an advanced fluency in Python scripting, operating system process iteration (`psutil`), asynchronous web socket emission configurations (`Flask-SocketIO`), and low-level SQL cursor management techniques. It serves as the definitive reference manual for debugging or modifying the core execution logic.

2. MODULE OVERVIEW

To maximize execution speed and eliminate inter-module dependency latency, the application logic is deliberately consolidated into a single monolithic Python execution script. This script is structurally segmented into discrete functional blocks: global variable instantiation, database initialization, synchronous SRE logic, asynchronous RPA logic, and WSGI server callbacks. The application leverages the Werkzeug WSGI server to handle inbound web traffic while aggressively isolating the blocking UI automation code into a daemonized thread. This programmatic separation prevents the long-running automation algorithms from starving the web server of CPU cycles.

3. ALGORITHMIC DESIGN

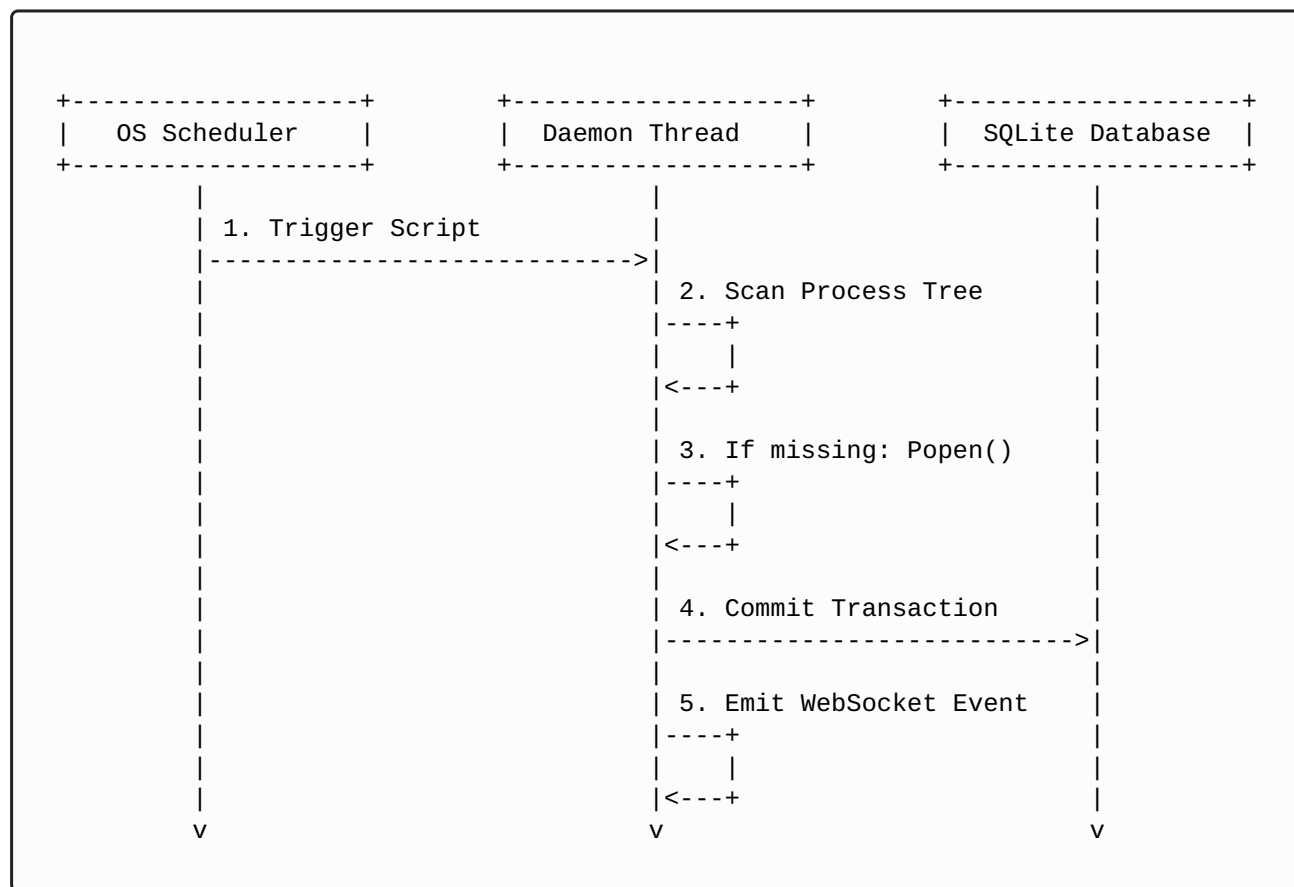
3.1 Thread Configuration

The core execution divergence is implemented utilizing Python's native `threading.Thread` module. When the `__main__` block executes, it spawns a secondary worker thread: `threading.Thread(target=bot_loop, daemon=True).start()`. The explicit parameter `daemon=True` is a critical architectural requirement; it ensures that if the primary Flask server thread encounters a fatal exception or receives a manual SIGINT (CTRL+C), the background automation loop is immediately terminated by the Python interpreter, preventing unkillable zombie processes.

3.2 Daemon Process Flow

The internal function execution within the daemon thread follows a strict procedural hierarchy. The loop enters a `while True` state. Within this cycle, `monitor_and_heal()` is invoked to actively evaluate the OS process tree. Following health verification, `run_retail_automation()` generates random floating-point values. A localized `sqlite3` cursor is opened to execute parameterized `INSERT INTO` statements, followed immediately by `conn.commit()`. Following the database lock release, the `socketio.emit()` method serializes the updated dictionary state to the client over the TCP socket. Finally, `focus_window()` invokes the OS API to locate the target application, followed by blocking `pyautogui.write()` operations.

Figure 3.1: Sequential Daemon and Event Emission Flowchart



3.3 Error Handling Failsafes

Physical desktop automation via the `pyautogui` library is inherently volatile, as the Windows OS frequently rejects programmatic focus requests via the Desktop Window Manager (DWM). To mitigate fatal crashes, the `focus_window(app_name)` function implements a rigid `try/except` failsafe block. When a substring match is found in the active window title list, it wraps the `.activate()` method inside the exception handler. If the OS rejects the activation request, the `except` block catches it silently, allowing the bot loop to bypass the current automation cycle and continue execution rather than halting the entire Python script.

4. FUNCTION & CLASS CATALOGUE

The programmatic implementation relies on specific modular functions to achieve required execution constraints while maintaining absolute code readability. Each function is strictly scoped to perform a singular, deterministic task within the automation loop. This modularity ensures that future developers can update the parsing algorithms without disrupting the networking emission blocks.

The **init_db()** function is responsible for schema validation. Upon startup, it establishes a synchronous connection to the database file and executes a ``CREATE TABLE IF NOT EXISTS`` statement. This ensures that the application never attempts to insert data into a non-existent table, which would trigger a fatal ``OperationalError`` causing an immediate shutdown.

The **monitor_and_heal()** algorithm handles the core Service Reliability Engineering logic. It utilizes the ``psutil`` library to scan active hardware threads. Instead of allocating memory for a massive array of active processes, the algorithm utilizes an optimized Python generator expression: ``if "calc.exe" not in (p.name().lower() for p in psutil.process_iter()):``. If the target string is absent, it immediately executes ``subprocess.Popen()`` to force a localized application reboot without human intervention.

The **run_retail_automation()** function serves as the central orchestrator for transaction synthesis. It leverages the ``random`` module to generate constrained integers and floats, calculates the final receipt total, and manages the localized SQLite cursor connections. It is also exclusively responsible for mutating the global ``web_data`` dictionary prior to triggering the socket emission.

5. DATABASE SCHEMA & EXECUTION

5.1 Table Definitions

The schema relies entirely on Python's native `sqlite3` library. The DDL statement executed upon boot is: `CREATE TABLE IF NOT EXISTS transactions (txn_id TEXT PRIMARY KEY, customer TEXT, qty INTEGER, price REAL, total REAL, timestamp TEXT)`. Transaction variables are synthesized using core libraries, with the `client_id` synthesized using f-string concatenation: `f"TXN-{random.randint(1000, 9999)}"`. The floating-point mathematics for pricing are explicitly bounded using `round(value, 2)` to eliminate floating-point arithmetic pollution within the REAL columns.

5.2 SQL Execution Patterns

To maximize query execution speed, the script circumvents heavy Object-Relational Mappers (ORMs). Database cursors are aggressively instantiated, executed, and closed (`conn.close()`) within a singular function scope during every loop iteration. The parameterization tuple is strictly enforced: `c.execute("INSERT INTO transactions VALUES (?, ?, ?, ?, ?, ?)", (client_id, name, quantity, avg_price, total, timestamp))`. This specific scoping and parameterization prevents "Database is Locked" (`sqlite3.OperationalError`) exceptions that plague persistent, multi-threaded SQLite connections.

6. NETWORK & HARDWARE INTERFACES

The system requires robust exception handling when interfacing with volatile external networks and restrictive hardware I/O boundaries. The outbound alerting mechanism is implemented within the `send_alert(message)` function, utilizing the external `requests` library to POST structured JSON payloads to a remote Discord server webhook. This establishes a secondary failure mechanism for system administrators.

Recognizing that external networks are inherently volatile and subject to DNS failures or timeout limits, the `requests.post()` call is entirely wrapped in a bare `try...except:` block. This explicit error handling is critical: if the host machine drops its internet connection, or the remote API experiences downtime, the resulting `ConnectionError` is silently trapped. This allows the retail automation bot to print an offline warning to the local terminal while continuing to process physical POS transactions uninterrupted. Without this failsafe, a simple Wi-Fi dropout would fatally crash the entire daemon.

For hardware interfaces, the `pyautogui` library interacts directly with the OS peripheral drivers. A `pyautogui.FailSafeException` is mitigated by artificially injecting `time.sleep()` commands between keystrokes to prevent buffer overruns during the physical receipt typing phase, ensuring the Windows Desktop Manager has sufficient cycles to render text.

7. VARIABLE & STATE MUTABILITY

State mutability is highly localized within the script to prevent memory leaks and variable corruption across parallel threads. The global `web_data` dictionary explicitly defines variable types upon script instantiation (e.g., initializing integers as `0`, floats as `0.0`, and strings as `--`). This rigid definition prevents critical type-coercion errors during the automatic JSON serialization phase executed by the SocketIO emitter, which could crash the frontend DOM renderer if improperly typed.

Within the `run_retail_automation()` scope, the script uses the Python `global web_data` directive to authorize the in-place mutation of these existing keys (e.g., executing `web_data["total_revenue"] += total`). This exact operational pattern maintains a continuous, stateful application session directly in the Python runtime RAM. By utilizing RAM as the definitive state cache, the script completely bypasses the need for repetitive, latency-heavy database `SELECT` polling to update the frontend dashboard, vastly accelerating execution cycles.

8. MEMORY OPTIMIZATION

On the presentation layer, the low-level rendering strategy focuses on strict JavaScript array manipulation to actively restrict browser RAM consumption over extended operational lifecycles. The Chart.js implementation pushes the `latest_total` payload directly into an active coordinate array each time a WebSocket payload is received, avoiding total refresh states.

To ensure the browser tab does not suffer memory exhaustion or DOM lag if the kiosk runs unattended for days, an algorithmic garbage collection check is implemented within the `socket.on` listener. Once the `data.labels.length` exceeds a hardcoded limit of 15 indices, the script triggers the `.shift()` array method. This aggressively discards the oldest data point from active memory before rendering the new graphical frame, creating a lightweight sliding window that completely eliminates client-side memory bloat.

9. RUNTIME SECURITY CONSTRAINTS

The application is fortified at the runtime execution level to prevent hostile intrusion. The Flask server initialization call is hardcoded to `app.run(debug=False)`. This explicitly disables the Werkzeug interactive debugger, a paramount security mandate to prevent arbitrary remote code execution (RCE) via the browser console in live kiosk environments.

Furthermore, thread synchronization issues related to the Global Interpreter Lock (GIL) are natively resolved by assigning `async_mode='threading'` to the SocketIO configuration. This ensures that the web server respects standard library parallelism rather than attempting unsafe asynchronous context switching across the database cursor boundaries.

10. REFERENCES

1. Python Official Documentation - sqlite3: <https://docs.python.org/3/library/sqlite3.html>
2. Python Official Documentation - threading: <https://docs.python.org/3/library/threading.html>
3. Flask-SocketIO Core API Documentation: <https://flask-socketio.readthedocs.io/>
4. psutil (Process and System Utilities) Execution API: <https://psutil.readthedocs.io/>